

M06 - Git + GitHub + Linux/WSL Engineering Foundations

13-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Work safely in Linux/WSL, version projects with Git/GitHub and hand off reproducible repositories.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M00 setup discipline and M04 basic scripting habits. You should be comfortable locating project files and using a terminal carefully.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE06-01 - Verify WSL2 and choose a safe Linux workspace	5
DE06-02 - Linux navigation and safe file operations	7
DE06-03 - Pipes, redirection, command history and inspection	9
DE06-04 - Permissions, packages, sudo and command safety	11
DE06-05 - Git state: init, status, add, commit, diff and log	13
DE06-06 - Branches, merges and real conflict resolution	15
DE06-07 - GitHub remotes: push, fetch, pull and tracking	17
DE06-08 - Pull requests and review workflow	19
DE06-09 - Python virtual environments inside a repository	21
DE06-10 - .gitignore, secrets and repository hygiene	23
DE06-11 - restore, revert, reflog and safe history repair	25
DE06-12 - Shell scripts, exit codes and environment-driven commands	27
DE06-13 - Repository handoff and clean-clone reproducibility	29

Prerequisite, tool and outcome map

ASSUMED

Complete M00 setup discipline and M04 basic scripting habits. You should be comfortable locating project files and using a terminal carefully.

MODULE PURPOSE

Work safely in Linux/WSL, version projects with Git/GitHub and hand off reproducible repositories.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE06-01 - Verify WSL2 and choose a safe Linux workspace

1. Why it matters

Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory.

3. Prerequisite / what you already know

Complete M00 setup discipline and M04 basic scripting habits. You should be comfortable locating project files and using a terminal carefully.

4. Picture it in your head

WSL gives you a Linux room inside Windows. Choose one stable workspace path and know which filesystem you are using so tools do not silently read a different copy.

5. New words before we use them

WSL gives a Windows machine a real Linux user space. First inspect what already exists.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(learn.microsoft.com\)](https://learn.microsoft.com/en-us/windows/wsl/install)

URL: <https://learn.microsoft.com/en-us/windows/wsl/install>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

PowerShell: `wsl --status;`

`wsl -l -v Ubuntu: whoami;`

`pwd;`

`uname -a;`

`mkdir -p ~/de-course/m06` Verify your current WSL state, identify the distro and WSL version, enter the distro, then create `~/de-course/m06/lesson_lab`.

8. Expected result / what you should see

You can show the intended output/state for Verify WSL2 and choose a safe Linux workspace, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

This works because verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory is made explicit before the result is accepted. Your independent check is: For Verify WSL2 and choose a safe Linux workspace: record the actual before/after state and verify the lesson requirement with an independent control. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If `wsl --install` shows help, inspect existing distros first. Use current Microsoft troubleshooting instead of random manual feature changes. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: WSL gives you a Linux room inside Windows. 3. Run the earliest independent control: For Verify WSL2 and choose a safe Linux workspace: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Close the video/book. Prove which terminal you are in, show your Linux home path, create `~/de-course/m06/recheck`, and explain why re-running installation is unnecessary when WSL is already healthy. Validate result A usable distro is visible; Linux commands run inside it; the workspace is under Linux home; your note distinguishes PowerShell/WSL-management commands from Linux commands.

13. Validate your result

For Verify WSL2 and choose a safe Linux workspace: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: You can show the intended output/state for Verify WSL2 and choose a safe Linux workspace, name its grain or boundary where relevant, and point to one independent check that confirms it.

14. Teach it back in your own words

Without notes, explain Verify WSL2 and choose a safe Linux workspace in your own words. Start with the purpose: Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory. Then name one realistic failure mode and explain how you would prove the result using this check: For Verify WSL2 and choose a safe Linux workspace: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Verify WSL2 and choose a safe Linux workspace with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-02.

DE06-02 - Linux navigation and safe file operations

1. Why it matters

Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files.

3. Prerequisite / what you already know

Prerequisite: DE06-01 mastered with saved evidence.

4. Picture it in your head

The shell is a map plus verbs: pwd tells where you are, ls shows nearby items, cd moves, and cp/mv/rm change files. Always confirm location before a destructive verb.

5. New words before we use them

Your current directory changes the meaning of every relative path. Before a destructive file command, establish location and target with pwd and ls.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([learn.microsoft.com](https://learn.microsoft.com/en-us/windows/wsl/install))

URL: <https://learn.microsoft.com/en-us/windows/wsl/install>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`cd ~/de-course/m06/lesson_lab` `pwd` `mkdir -p sandbox/input sandbox/output` `touch sandbox/input/note.txt` `cp sandbox/input/note.txt sandbox/output/copy.txt` `mv sandbox/output/copy.txt sandbox/output/moved.txt` `ls -la sandbox/output` `rm sandbox/output/moved.txt` Build the sandbox tree, inspect source and destination before every cp/mv/rm, and narrate exactly what changes.

8. Expected result / what you should see

You can show the intended output/state for Linux navigation and safe file operations, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The setup action is trustworthy because the actual file/system state is checked after the operation; successful clicking or command execution alone is not treated as proof. Your independent check is: For Linux navigation and safe file operations: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If location or target is uncertain, stop and run `pwd` plus `ls -la`. Never use `rm -rf` to resolve uncertainty. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: The shell is a map plus verbs: `pwd` tells where you are, `ls` shows nearby items, `cd` moves, and `cp/mv/rm` change files. 3. Run the earliest independent control: For Linux navigation and safe file operations: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build a different nested tree. Move one named text file through two directories; before deleting it, save `pwd` and `ls` evidence for the exact target directory. Validate result Only the intended sandbox object changes and you can explain absolute vs relative paths plus source vs destination.

13. Validate your result

For Linux navigation and safe file operations: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

14. Teach it back in your own words

Without notes, explain Linux navigation and safe file operations in your own words. Start with the purpose: Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files. Then name one realistic failure mode and explain how you would prove the result using this check: For Linux navigation and safe file operations: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Linux navigation and safe file operations with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-03.

DE06-03 - Pipes, redirection, command history and inspection

1. Why it matters

Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code.

3. Prerequisite / what you already know

Prerequisite: DE06-02 mastered with saved evidence.

4. Picture it in your head

A pipe connects one command's output to the next command's input; redirection sends output to a file. Picture commands as small boxes connected by arrows.

5. New words before we use them

A pipe sends standard output from one command to the next. > replaces a file; >> appends.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(learn.microsoft.com\)](https://learn.microsoft.com/en-us/windows/wsl/install)

URL: <https://learn.microsoft.com/en-us/windows/wsl/install>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
head data/raw/orders.csv grep '
```

```
Completed'
```

```
data/raw/orders.csv | wc -l printf '
```

```
east\nwest\neast\n'
```

```
| sort | uniq -c python3 src/pipeline.py >
```

```
evidence.txt echo $? echo '
```

```
second note'
```

```
>
```

```
>
```

evidence.txt history | tail Count a known status with grep|wc, create a frequency count with sort|uniq -c, redirect pipeline output, inspect the exit code, then append a second line without erasing the first.

8. Expected result / what you should see

You can show the intended output/state for Pipes, redirection, command history and inspection, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

This works because use pipes, redirection, history and small inspection commands to understand data and command output before writing more code is made explicit before the result is accepted. Your independent check is: For Pipes, redirection, command history and inspection: Run the changed pipeline command twice: once to screen and once with redirection. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

Run each stage separately. Do not add more pipes while you are unsure which stage changed the result. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A pipe connects one command's output to the next command's input; redirection sends output to a file. 3. Run the earliest independent control: For Pipes, redirection, command history and inspection: Run the changed pipeline command twice: once to screen and once with redirection. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Use a different status/value from the bundled CSV to build one pipeline and save the result to evidence/inspection.txt. Validate result You can predict where output goes, demonstrate > versus >>, and break a wrong pipeline into stages to find the defect.

13. Validate your result

For Pipes, redirection, command history and inspection: Run the changed pipeline command twice: once to screen and once with redirection. Compare exit status plus the exact saved file contents, and use history to identify the command that produced the evidence.

14. Teach it back in your own words

Without notes, explain Pipes, redirection, command history and inspection in your own words. Start with the purpose: Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code. Then name one realistic failure mode and explain how you would prove the result using this check: For Pipes, redirection, command history and inspection: Run the changed pipeline command twice: once to screen and once with redirection.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Pipes, redirection, command history and inspection with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-04.

DE06-04 - Permissions, packages, sudo and command safety

1. Why it matters

Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts.

3. Prerequisite / what you already know

Prerequisite: DE06-03 mastered with saved evidence.

4. Picture it in your head

Permissions are door locks for owner/group/others; sudo temporarily uses administrative authority. Use the smallest privilege needed and know which command receives it.

5. New words before we use them

sudo raises privilege for one command; it is not a generic make-it-work prefix. chmod changes permission bits.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(learn.microsoft.com\)](https://learn.microsoft.com/en-us/windows/wsl/install)

URL: <https://learn.microsoft.com/en-us/windows/wsl/install>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`ls -l scripts/run_pipeline.sh chmod u+x scripts/run_pipeline.sh ls -l scripts/run_pipeline.sh` # system package metadata only when needed: `sudo apt update` Inspect permissions, add execute permission only to the bundled training script, and explain why `sudo python3 src/pipeline.py` is suspicious for a normal user-owned project.

8. Expected result / what you should see

You can show the intended output/state for Permissions, packages, sudo and command safety, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

This works because use permissions, package commands and sudo with least privilege and reject dangerous shortcuts is made explicit before the result is accepted. Your independent check is: For Permissions, packages, sudo and command safety: Inspect file permissions and the exact command before/after the change. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

Permission denied? Inspect owner, path and permission bits first. Do not jump straight to sudo or chmod 777. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Permissions are door locks for owner/group/others; sudo temporarily uses administrative authority. 3. Run the earliest independent control: For Permissions, packages, sudo and command safety: Inspect file permissions and the exact command before/after the change. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Classify the bundled command-safety worksheet as SAFE / NEEDS EXPLANATION / REJECT and justify target, privilege and consequence. Validate result Before executing a privileged/destructive command you can name the target and expected change; no chmod 777, blind recursive deletion or sudo-as-default is used.

13. Validate your result

For Permissions, packages, sudo and command safety: Inspect file permissions and the exact command before/after the change. Prove one permitted action succeeds, one intentionally disallowed action fails as expected, and no broader sudo change was made than necessary.

14. Teach it back in your own words

Without notes, explain Permissions, packages, sudo and command safety in your own words. Start with the purpose: Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts. Then name one realistic failure mode and explain how you would prove the result using this check: For Permissions, packages, sudo and command safety: Inspect file permissions and the exact command before/after the change.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Permissions, packages, sudo and command safety with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-05.

DE06-05 - Git state: init, status, add, commit, diff and log

1. Why it matters

Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it.

3. Prerequisite / what you already know

Prerequisite: DE06-04 mastered with saved evidence.

4. Picture it in your head

Git has three useful areas: working tree -> staging area -> commit history. status tells where changes are; add selects the snapshot; commit records it.

5. New words before we use them

Git has distinct states. Editing changes the working tree; git add proposes content for the next snapshot; git commit records the staged snapshot.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 0:01:04–0:05:41 + 0:14:41–0:20:20

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=mAFoRnOfHs))

URL: <https://www.youtube.com/watch?v=mAFoRnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
git init git status git add README.md git diff --staged git commit -m "docs: establish project purpose"
```

git log --oneline --decorate -5 Initialize the lesson repository, make two focused changes, stage only one, inspect both unstaged and staged diffs, commit, then inspect history.

8. Expected result / what you should see

You can show the intended output/state for Git state: init, status, add, commit, diff and log, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The operation is trustworthy because related changes cross the commit boundary together; a before/after state check proves partial updates are not being mistaken for success. Your independent check is: For Git state: init, status, add, commit, diff and log: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If you are unsure what is staged, do not commit. Use git status, git diff and git diff --staged first. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Git has three useful areas: working tree -> staging area -> commit history. 3. Run the earliest independent control: For Git state: init, status, add, commit, diff and log: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create three simultaneous states: one committed file, one staged change and one unstaged change. Explain each line of git status without changing state while explaining. Validate result You can point to the working-tree, index and repository states and use diff/log evidence to prove what will or did enter a commit.

13. Validate your result

For Git state: init, status, add, commit, diff and log: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

14. Teach it back in your own words

Without notes, explain Git state: init, status, add, commit, diff and log in your own words. Start with the purpose: Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it. Then name one realistic failure mode and explain how you would prove the result using this check: For Git state: init, status, add, commit, diff and log: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Git state: init, status, add, commit, diff and log with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-06.

DE06-06 - Branches, merges and real conflict resolution

1. Why it matters

Create focused branches, merge work and resolve a real text conflict while preserving intended content. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Create focused branches, merge work and resolve a real text conflict while preserving intended content.

3. Prerequisite / what you already know

Prerequisite: DE06-05 mastered with saved evidence.

4. Picture it in your head

Branches are parallel timelines pointing to commits. A merge combines timelines; a conflict is Git asking you to choose the correct final content where automatic combination is unsafe.

5. New words before we use them

A branch is a movable name pointing into commit history. A merge combines histories.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 0:40:58–0:52:13

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=mAFoROnOfHs)

URL: <https://www.youtube.com/watch?v=mAFoROnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`git switch -c feature/docs # edit and commit git switch main # conflicting edit and commit git merge feature/docs # resolve file intentionally git add docs/architecture.md git commit git status` Use the bundled conflict fixture to create two incompatible edits to the same line, merge them, resolve to the required combined meaning, and record the final graph.

8. Expected result / what you should see

You can show the intended output/state for Branches, merges and real conflict resolution, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For Branches, merges and real conflict resolution: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

Use git status to list unmerged paths and inspect the file. If confused, stop before destructive reset; the lesson review explains safe abort/retry options. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Branches are parallel timelines pointing to commits. 3. Run the earliest independent control: For Branches, merges and real conflict resolution: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a fresh conflict in a different file. Resolve it without deleting unrelated content and explain why keeping only one side blindly is unsafe. Validate result No conflict markers remain; intended content from both changes is preserved where required; git status is clean; history shows the merge.

13. Validate your result

For Branches, merges and real conflict resolution: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation.

14. Teach it back in your own words

Without notes, explain Branches, merges and real conflict resolution in your own words. Start with the purpose: Create focused branches, merge work and resolve a real text conflict while preserving intended content. Then name one realistic failure mode and explain how you would prove the result using this check: For Branches, merges and real conflict resolution: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Branches, merges and real conflict resolution with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-07.

DE06-07 - GitHub remotes: push, fetch, pull and tracking

1. Why it matters

Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits.

3. Prerequisite / what you already know

Prerequisite: DE06-06 mastered with saved evidence.

4. Picture it in your head

Local Git is your notebook; a remote is another copy with shared history. fetch updates what you know, pull integrates remote changes, and push publishes your local commits.

5. New words before we use them

A remote name such as origin is a local configuration pointing to another repository. push sends commits/refs; fetch downloads remote state without merging it; pull fetches and then integrates according to configuration.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 0:16:04–0:20:20 + 0:55:50–1:00:24

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=mAFoRnOfHs)

URL: <https://www.youtube.com/watch?v=mAFoRnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`git remote -v git push -u origin main git fetch origin git branch -vv git pull --ff-only` Use the bundled local bare remote first: push main, clone it into a second directory, make a remote-side commit, fetch from the first clone, inspect the remote-tracking branch, then integrate deliberately.

8. Expected result / what you should see

You can show the intended output/state for GitHub remotes: push, fetch, pull and tracking, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For GitHub remotes: push, fetch, pull and tracking: Use `git status/diff/log` (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If histories diverge, inspect git status, branch -vv and log --all --graph before choosing merge/rebase/reset. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Local Git is your notebook; a remote is another copy with shared history. 3. Run the earliest independent control: For GitHub remotes: push, fetch, pull and tracking: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Repeat with a changed remote update. Before pull, prove what fetch changed and what it did not change. Validate result You can distinguish local main, origin/main and the remote repository and explain push vs fetch vs pull.

13. Validate your result

For GitHub remotes: push, fetch, pull and tracking: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

14. Teach it back in your own words

Without notes, explain GitHub remotes: push, fetch, pull and tracking in your own words. Start with the purpose: Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits. Then name one realistic failure mode and explain how you would prove the result using this check: For GitHub remotes: push, fetch, pull and tracking: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for GitHub remotes: push, fetch, pull and tracking with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-08.

DE06-08 - Pull requests and review workflow

1. Why it matters

Use a feature branch and pull request as a review contract, then synchronize local main after merge. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Use a feature branch and pull request as a review contract, then synchronize local main after merge.

3. Prerequisite / what you already know

Prerequisite: DE06-07 mastered with saved evidence.

4. Picture it in your head

Local Git is your notebook; a remote is another copy with shared history. fetch updates what you know, pull integrates remote changes, and push publishes your local commits.

5. New words before we use them

A pull request is a proposed branch-to-branch change with a diff, discussion and review history. The PR does not replace Git commits.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 1:16:48–1:19:57

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=mAFoROnOfHs)

URL: <https://www.youtube.com/watch?v=mAFoROnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`git switch -c feature/readme # edit/test/commit/push # On GitHub: open PR ->`

`inspect Files changed ->`

`merge git switch main git pull --ff-only` Prepare a small branch, write a PR-style summary from the diff, and use a local review checklist. If a GitHub account is available, perform the real PR;

otherwise save the exact handoff evidence for later account smoke.

8. Expected result / what you should see

You can show the intended output/state for Pull requests and review workflow, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For Pull requests and review workflow: Use `git status/diff/log` (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If the PR contains unrelated files, do not explain them away. Repair the branch/commits so the diff itself becomes reviewable. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Local Git is your notebook; a remote is another copy with shared history. 3. Run the earliest independent control: For Pull requests and review workflow: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a second tiny branch and produce a reviewable PR packet without copying the first summary. Validate result Diff matches the stated scope; validation is recorded; after merge/simulation local main contains the intended commit and the feature branch is no longer the active work location.

13. Validate your result

For Pull requests and review workflow: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

14. Teach it back in your own words

Without notes, explain Pull requests and review workflow in your own words. Start with the purpose: Use a feature branch and pull request as a review contract, then synchronize local main after merge. Then name one realistic failure mode and explain how you would prove the result using this check: For Pull requests and review workflow: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Pull requests and review workflow with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-09.

DE06-09 - Python virtual environments inside a repository

1. Why it matters

Create a disposable Python virtual environment in the repository workspace while keeping it out of version control. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Create a disposable Python virtual environment in the repository workspace while keeping it out of version control.

3. Prerequisite / what you already know

Prerequisite: DE06-08 mastered with saved evidence.

4. Picture it in your head

A virtual environment is a project-specific tool cabinet for Python packages, preventing one project's dependency versions from silently changing another project.

5. New words before we use them

A virtual environment isolates a project interpreter/package set. It is disposable, not a deliverable.

6. Useful video / source

Source: Luke Barousse — virtual environments / dependencies

Assigned scope: 6:10:49–6:27:15

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
python3 -m venv .venv source .venv/bin/activate python -c "
import sys;
print(sys.executable)"
deactivate git status --ignored --short Create .venv, activate it, prove the Python path is inside the environment, deactivate it and verify Git ignores the directory.
```

8. Expected result / what you should see

You can show the intended output/state for Python virtual environments inside a repository, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For Python virtual environments inside a repository: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If activation/path is confusing, inspect which python/python3 and sys.executable. Never solve it by committing the environment directory. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A virtual environment is a project-specific tool cabinet for Python packages, preventing one project's dependency versions from silently changing another project. 3. Run the earliest independent control: For Python virtual environments inside a repository: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Delete only the disposable .venv, recreate it from saved instructions and rerun the bundled pipeline. Validate result Project behavior survives environment recreation and Git does not track .venv.

13. Validate your result

For Python virtual environments inside a repository: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Python virtual environments inside a repository in your own words. Start with the purpose: Create a disposable Python virtual environment in the repository workspace while keeping it out of version control. Then name one realistic failure mode and explain how you would prove the result using this check: For Python virtual environments inside a repository: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Python virtual environments inside a repository with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-10.

DE06-10 - .gitignore, secrets and repository hygiene

1. Why it matters

Keep secrets, virtual environments, caches and generated runtime outputs out of Git while preserving reproducible examples/configuration. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Keep secrets, virtual environments, caches and generated runtime outputs out of Git while preserving reproducible examples/configuration.

3. Prerequisite / what you already know

Prerequisite: DE06-09 mastered with saved evidence.

4. Picture it in your head

Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. The secret belongs in configuration, not source code/logs.

5. New words before we use them

Ignore rules prevent untracked matching files from entering normal Git status, but they do not untrack content already committed. A real leaked credential must be revoked/rotated; deleting its current line is not enough.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: Use repo-hygiene / .gitignore portion; Teaching Book adds secret safety

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=mAFoRnOfHs)

URL: <https://www.youtube.com/watch?v=mAFoRnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
printf '
.env/.env\n__pycache__\nnevidence/runtime\n'
>
.gitignore printf '
API_URL=http://localhost:9999\nTOKEN=replace_me\n'
>
.env.example git check-ignore -v .env .env/ || true git status --ignored --short Create the ignore file and example config,
generate fake runtime files, and prove they are ignored while .env.example remains trackable.
```

8. Expected result / what you should see

You can show the intended output/state for .gitignore, secrets and repository hygiene, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For .gitignore, secrets and repository hygiene: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If a real credential ever reaches Git history, stop normal practice, revoke/rotate it and follow provider/GitHub cleanup guidance. Do not continue as though deletion solved exposure. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. 3. Run the earliest independent control: For .gitignore, secrets and repository hygiene: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Introduce a fake secret file into a changed sandbox, catch it before commit using status/diff/check-ignore, and repair the ignore boundary. Validate result No real secret is used; runtime junk is untracked; reproducible examples are committed; you can explain why ignore rules do not erase history.

13. Validate your result

For .gitignore, secrets and repository hygiene: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

14. Teach it back in your own words

Without notes, explain .gitignore, secrets and repository hygiene in your own words. Start with the purpose: Keep secrets, virtual environments, caches and generated runtime outputs out of Git while preserving reproducible examples/configuration. Then name one realistic failure mode and explain how you would prove the result using this check: For .gitignore, secrets and repository hygiene: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for .gitignore, secrets and repository hygiene with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-11.

DE06-11 - restore, revert, reflog and safe history repair

1. Why it matters

Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history.

3. Prerequisite / what you already know

Prerequisite: DE06-10 mastered with saved evidence.

4. Picture it in your head

A pipe connects one command's output to the next command's input; redirection sends output to a file. Picture commands as small boxes connected by arrows.

5. New words before we use them

restore operates on working-tree/index content; revert creates a new commit that inverses an earlier commit, making it suitable for shared history; reflog records local reference movements and can help recover a commit name you lost. reset can be useful, but destructive forms are not a default recovery strategy in this course.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: Use history/repair portion; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=mAFoROnOfHs))

URL: <https://www.youtube.com/watch?v=mAFoROnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
git restore path/to/file git revert <
bad-commit>
git reflog --oneline -10 git show <
recovered-commit>
```

Make and restore an unstaged mistake, revert a committed mistake, then move a branch reference in a disposable lab and locate the prior commit via reflog.

8. Expected result / what you should see

You can show the intended output/state for restore, revert, reflog and safe history repair, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For restore, revert, reflog and safe history repair: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

Before any recovery command, classify the mistake: uncommitted content, staged content, committed/shared history, or lost reference. Then choose the smallest safe tool. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A pipe connects one command's output to the next command's input; redirection sends output to a file. 3. Run the earliest independent control: For restore, revert, reflog and safe history repair: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Given three changed incidents, select restore/revert/reflog and justify the choice before running anything. Validate result. Final content is correct, history is explainable, and no shared-history rewrite is used as a shortcut.

13. Validate your result

For restore, revert, reflog and safe history repair: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

14. Teach it back in your own words

Without notes, explain restore, revert, reflog and safe history repair in your own words. Start with the purpose: Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history. Then name one realistic failure mode and explain how you would prove the result using this check: For restore, revert, reflog and safe history repair: Use git status/diff/log (and branch graph where relevant) to prove the exact before/after repository state; a clean working tree should be explainable.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for restore, revert, reflog and safe history repair with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-12.

DE06-12 - Shell scripts, exit codes and environment-driven commands

1. Why it matters

Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes.

3. Prerequisite / what you already know

Prerequisite: DE06-11 mastered with saved evidence.

4. Picture it in your head

A shell script is a repeatable command sequence. Exit code 0 means success by convention; non-zero signals failure so other tools can stop instead of continuing blindly.

5. New words before we use them

A shell script turns a command sequence into a repeatable interface. Quote variable expansions, validate required inputs before use, return exit code 0 only for success and nonzero for expected failure states.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(learn.microsoft.com\)](https://learn.microsoft.com/en-us/windows/wsl/install)

URL: <https://learn.microsoft.com/en-us/windows/wsl/install>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
#!/usr/bin/env bash set -u INPUT=${INPUT_FILE:-data/raw/orders.csv} [[ -f "$INPUT" ]] || { echo "missing: $INPUT" >&2; exit 2; } python3 src/pipeline.py "$INPUT" rc=$? exit "$rc" Run the bundled script with its default input, then with INPUT_FILE set to a valid alternative and a missing file. Record stdout/stderr and exit codes.
```

8. Expected result / what you should see

You can show the intended output/state for Shell scripts, exit codes and environment-driven commands, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

This works because write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes is made explicit before the result is accepted. Your independent check is: For Shell scripts, exit codes and environment-driven commands: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected c. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If behavior differs between interactive command and script, print only non-secret configuration/path state and inspect exit codes step-by-step. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A shell script is a repeatable command sequence. 3. Run the earliest independent control: For Shell scripts, exit codes and environment-driven commands: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a changed runner for the fresh-retest workspace with one required environment variable and two distinct nonzero failure codes. Validate result Success returns 0; missing/invalid inputs return documented nonzero codes; paths containing spaces are safely quoted; no credential value is printed.

13. Validate your result

For Shell scripts, exit codes and environment-driven commands: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Shell scripts, exit codes and environment-driven commands in your own words. Start with the purpose: Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes. Then name one realistic failure mode and explain how you would prove the result using this check: For Shell scripts, exit codes and environment-driven commands: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected c.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Shell scripts, exit codes and environment-driven commands with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-13.

DE06-13 - Repository handoff and clean-clone reproducibility

1. Why it matters

Prove that another machine/user can recreate and run the project from a clean clone without hidden local state. This matters because shell and Git mistakes can make a project irreproducible, overwrite work or leave no trustworthy history of what changed.

2. What you are learning

Prove that another machine/user can recreate and run the project from a clean clone without hidden local state.

3. Prerequisite / what you already know

Prerequisite: DE06-12 mastered with saved evidence.

4. Picture it in your head

A clean clone is the real handoff test: if a new folder can be cloned, configured from documented steps, run, and validated, the repository is reproducible rather than dependent on your machine.

5. New words before we use them

A project is not reproducible because it works in the author directory. A clean-clone test deliberately starts without untracked files, local virtual environments or shell history.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: Use clone/push/pull portions; then do clean-clone handoff

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=mAFoRnOfHs)

URL: <https://www.youtube.com/watch?v=mAFoRnOfHs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
git clone <
remote>
```

```
clean-check cd clean-check python3 -m venv .venv source .venv/bin/activate ./scripts/run_pipeline.sh git status --short
```

Push/prepare the bundled repository, clone it to a new directory, recreate its environment, run the pipeline and prove the working tree remains clean except for explicitly ignored outputs.

8. Expected result / what you should see

You can show the intended output/state for Repository handoff and clean-clone reproducibility, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The change is trustworthy because repository state is observable before and after the operation, and the commit/diff/history provides an auditable explanation of exactly what changed. Your independent check is: For Repository handoff and clean-clone reproducibility: Clone the repository into a brand-new folder, follow only the documented setup steps, run the required command/test, and confirm git status is clean afterward. The proof is observable shell/repository state and reproducibility from a clean workspace.

10. Common beginner mistakes

If the clean clone fails, treat that as a project defect. Identify the hidden dependency, add a safe reproducible instruction/configuration, commit it, clone again and retest. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A clean clone is the real handoff test: if a new folder can be cloned, configured from documented steps, run, and validated, the repository is reproducible rather than dependent on your machine. 3. Run the earliest independent control: For Repository handoff and clean-clone reproducibility: Clone the repository into a brand-new folder, follow only the documented setup steps, run the required command/test, and confirm git status is clean afterward. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Hand the changed-retest repository to a clean directory and reproduce it using only committed instructions. Record every missing assumption you discover and repair the repository rather than the clone by hand. Validate result A clean clone can be set up and run from documented files; no hidden absolute path or secret is required; validation totals match; repository state is explainable.

13. Validate your result

For Repository handoff and clean-clone reproducibility: Clone the repository into a brand-new folder, follow only the documented setup steps, run the required command/test, and confirm git status is clean afterward. Record any missing hidden dependency as a handoff defect.

14. Teach it back in your own words

Without notes, explain Repository handoff and clean-clone reproducibility in your own words. Start with the purpose: Prove that another machine/user can recreate and run the project from a clean clone without hidden local state. Then name one realistic failure mode and explain how you would prove the result using this check: For Repository handoff and clean-clone reproducibility: Clone the repository into a brand-new folder, follow only the documented setup steps, run the required command/test, and confirm git status is clean afterward.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Repository handoff and clean-clone reproducibility with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.